

ssize Should be Named count

Document number: P1764r0

Date: 2019-06-17

Audience: LEWG

Reply-to: Tony Van Eerd. count at forecode.com

I've been through the papers, found some stuff with bad names
It felt bad to be causing dev pain
In the standard you need stuff with good names
Cause there ain't no one for to help you explain
La la, la la-la la

After two days in the standard tome
My brain began to turn dead
After three days with the standard combed
I was looking at a lot of dread
And the story it told of those names in code
Made me sad to think of our devs

You see I've been through the papers, found some stuff with bad names
It felt bad to be causing dev pain
In the standard you need stuff with good names
Cause there ain't no one for to help you explain
La la, la la-la la

....

(ie *A Horse with No Name* by America)

Summary

To resolve (or at least temporarily quell) the signed/unsigned debate, P1227 (approved in Kona) proposes adding a member function `ssize()` to `span`, which is just like `size()`, but which returns a signed value. It also proposed adding `std::ssize(container)` for all standard containers, and `ssize()` member functions for all containers, not just `span`.

Bjarne's P1491 (and P1428) however notes:

[p1227R1] proposes to change `size()` in the ranges TS and for `span` to unsigned (making them bug compatible with the STL) and adding `ssize()` to all containers and range accessors

- embeds a type in a function name (and it makes me think of Parseltongue :-))
- leaves the wrong solution (IMO) with the better, more established, and simpler name

Thus this proposal - Use the name `count` instead of `ssize`

- does not embed type info
- is a better name than `ssize`
- is actually a better name than `size` (which is confused with `sizeof`)
- will eventually (20years?) become the established, simple, more intent-ful name

Current uses of `count()`

Functions named `count` can already be found in the standard.

Consistent/Compatible existing usage of "count"

- `AssociativeContainer::count(key)` returns the number of items with `key`. This is consistent with a function `AssociativeContainer::count()` returning the number of *all* the items.

Similarly, these Algorithms:

- `std::[range]::count(first, last, value [, projection])`
- `std::[range]::count_if(first,last, predicate [, projection])`
- `std::range::count(range, value [, projection])`
- `std::range::count_if(range, predicate [, projection])`

are consistent with a `std::count(container/range)` that would return the total number of elements of the container/range

(P.S. the above algorithms return signed values)

- Ranges also has `counted_iterator::count()` returning the "length" of the iterator (ie number of iterations to get to end). (also a signed value)
- `chrono::duration.count()` - number of ticks

All the above are consistent with using the name `count` for the number of items in a range or container.

Why We can't have nice things

`bitset<N>::count` returns the number of *on* bits. Not the size (ie not the number of total bits, both on and off).

We could deprecate `count()` on `bitset`, replace it with what it really does - `popcount` (ie "population count" if you ever wondered what `popcount` meant). Or `count_on`, etc.

And then wait 10-15 years, then add `int count() // returns number of elements` to `bitset`.

Corollary

Introduce `std::count_t` as the type returned by `count()`.

Currently, no one knows what type to use in a simple for loop. `int?` `size_t?` `ptrdiff_t?` `int64_t?` None of them suggest the intent of counting the cardinality of a container. `int` was suppose to be "the native size" of the architecture, but the move from 32bit to 64bit processors somehow left `int` behind, unfortunately. We embarassingly have no answer for how to count.